

Attribute (.DBF) API

The Attribute (DBF) API uses DBFHandle to represent a handle for access to one .dbf file. The contents of the DBFHandle are visible (see shapefil.h) but should be ignored by the application. It is intended that all information be accessed by API functions. Note that there should be exactly one record in the .dbf file for each shape in the .shp/.shx files. This constraint must be maintained by the application.

DBFOpen()

```
DBFHandle DBFOpen( const char * pszDBFFile, const char * pszAccess );
```

pszDBFFile: The name of the xBase (.dbf) file to access.

pszAccess: The fopen() style access string. At this time only "rb" (read-only binary) and "rb+" (read/write binary) should be used.

The DBFOpen() function should be used to establish access to an existing xBase format table file. The returned DBFHandle is passed to other access functions, and DBFClose() should be invoked to recover resources, and flush changes to disk when complete. The DBFCreate() function should be called to create new xBase files. As a convenience, DBFOpen() can be called with the name of a .shp or .shx file, and it will figure out the name of the related .dbf file.

DBFCreate()

```
DBFHandle DBFCreate( const char * pszDBFFile );
```

pszDBFFile: The name of the xBase (.dbf) file to create.

The DBFCreate() function creates a new xBase format file with the given name, and returns an access handle that can be used with other DBF functions. The newly created file will have no fields, and no records. Fields should be added with DBFAddField() before any records are written.

DBFGetFieldCount()

```
int DBFGetFieldCount( DBFHandle hDBF );
```

hDBF: The access handle for the file to be queried, as returned by DBFOpen(), or DBFCreate().

The DBFGetFieldCount() function returns the number of fields currently defined for the indicated xBase file.

DBFGetRecordCount()

```
int DBFGetRecordCount( DBFHandle hDBF );
```

hDBF: The access handle for the file to be queried, as returned by DBFOpen(), or DBFCreate().

The DBFGetRecordCount() function returns the number of records that exist on the xBase file currently. Note that with shape files one xBase record exists for each shape in the .shp/.shx files.

DBFGetFieldIndex()

```
int DBFGetFieldIndex( DBFHandle hDBF, const char *pszFieldName );
```

hDBF: The access handle for the file to be queried, as returned by DBFOpen(), or DBFCreate().

pszFieldName: Name of the field to search for.

Returns the index of the field matching this name, or -1 on failure. The comparison is case insensitive. However, lengths must match exactly.

DBFGetFieldInfo()

```
DBFFieldType DBFGetFieldInfo( DBFHandle hDBF, int iField, char * pszFieldName,
                               int * pnWidth, int * pnDecimals );
```

hDBF: The access handle for the file to be queried, as returned by DBFOpen(), or DBFCreate().

iField: The field to be queried. This should be a number between 0 and n-1, where n is the number of fields on the file, as returned by DBFGetFieldCount().

pszFieldName: If this pointer is not NULL the name of the requested field will be written to this location. The pszFieldName buffer should be at least 12 character in size in order to hold the longest possible field name of 11 characters plus a terminating zero character.

pnWidth: If this pointer is not NULL, the width of the requested field will be returned in the int pointed to by pnWidth. This is the width in characters.

pnDecimals: If this pointer is not NULL, the number of decimal places precision defined for the field will be returned. This is zero for integer fields, or non-numeric fields.

The DBFGetFieldInfo() returns the type of the requested field, which is one of the DBFFieldType enumerated values. As well, the field name, and field width information can optionally be returned. The field type returned does not correspond one to one with the xBase field types. For instance the xBase field type for Date will just be returned as being FTInteger.

```
typedef enum {
    FTString,           /* fixed length string field      */
    FTInteger,          /* numeric field with no decimals */
    FTDouble,           /* numeric field with decimals    */
    FTLogical,          /* logical field.                 */
    FTInvalid            /* not a recognised field type    */
} DBFFieldType;
```

DBFAddField()

```
int DBFAddField( DBFHandle hDBF, const char * pszFieldName,
                DBFFieldType eType, int nWidth, int nDecimals );
```

hDBF: The access handle for the file to be updated, as returned by DBFOpen(), or DBFCreate().

pszFieldName: The name of the new field. At most 11 character will be used. In order to use the xBase file in some packages it may be necessary to avoid some special characters in the field names such as spaces, or arithmetic operators.

eType: One of FTString, FTInteger or FTDouble in order to establish the type of the new field. Note that some valid xBase field types cannot be created such as date fields.

nWidth: The width of the field to be created. For FTString fields this establishes the maximum length of string that can be stored. For FTInteger this establishes the number of digits of the largest number that can be represented. For FTDouble fields this in combination with the nDecimals value establish the size, and precision of the created field.

nDecimals: The number of decimal places to reserve for FTDouble fields. For all other field types this should be zero. For instance with nWidth=7, and nDecimals=3 numbers would be formatted similarly to `123.456'.

The DBFAddField() function is used to add new fields to an existing xBase file opened with DBFOpen(), or created with DBFCreate().

The DBFAddField() return value is the field number of the new field, or -1 if the addition of the field failed.

DBFReadIntegerAttribute()

```
int DBFReadIntegerAttribute( DBFHandle hDBF, int iShape, int iField );
```

hDBF: The access handle for the file to be queried, as returned by DBFOpen(), or DBFCreate().

iShape: The record number (shape number) from which the field value should be read.

iField: The field within the selected record that should be read.

The DBFReadIntegerAttribute() will read the value of one field and return it as an integer. This can be used even with FTString fields, though the returned value will be zero if not interpretable as a number.

DBFReadDoubleAttribute()

```
double DBFReadDoubleAttribute( DBFHandle hDBF, int iShape, int iField );
```

hDBF: The access handle for the file to be queried, as returned by DBFOpen(), or DBFCreate().

iShape: The record number (shape number) from which the field value should be read.

iField: The field within the selected record that should be read.

The DBFReadDoubleAttribute() will read the value of one field and return it as a double. This can be used even with FTString fields, though the returned value will be zero if not interpretable as a number.

DBFReadStringAttribute()

```
const char *DBFReadStringAttribute( DBFHandle hDBF, int iShape, int iField );
```

hDBF: The access handle for the file to be queried, as returned by

DBFOpen(), or DBFCreate().

iShape: The record number (shape number) from which the field value should be read.

iField: The field within the selected record that should be read.

The DBFReadStringAttribute() will read the value of one field and return it as a string. This function may be used on any field type (including FTInteger and FTDoube) and will return the string representation stored in the .dbf file. The returned pointer is to an internal buffer which is only valid until the next DBF function call. It's contents may be copied with normal string functions such as strcpy(), or strdup(). If the TRIM_DBF_WHITESPACE macro is defined in shapefil.h (it is by default) then all leading and trailing space (ASCII 32) characters will be stripped before the string is returned.

DBFIsAttributeNULL()

```
int DBFIsAttributeNULL( DBFHandle hDBF, int iShape, int iField );
```

hDBF: The access handle for the file to be queried, as returned by DBFOpen(), or DBFCreate().

iShape: The record number (shape number) from which the field value should be read.

iField: The field within the selected record that should be read.

This function will return TRUE if the indicated field is NULL valued otherwise FALSE. Note that NULL fields are represented in the .dbf file as having all spaces in the field. Reading NULL fields will result in a value of 0.0 or an empty string with the other DBFRead*Attribute() functions.

DBFWriteIntegerAttribute

```
int DBFWriteIntegerAttribute( DBFHandle hDBF, int iShape, int iField,
                             int nFieldValue );
```

hDBF: The access handle for the file to be written, as returned by DBFOpen(), or DBFCreate().

iShape: The record number (shape number) to which the field value should be written.

iField: The field within the selected record that should be written.

nFieldValue: The integer value that should be written.

The DBFWriteIntegerAttribute() function is used to write a value to a numeric field (FTInteger, or FTDoube). If the write succeeds the value TRUE will be returned, otherwise FALSE will be returned. If the value is too large to fit in the field, it will be truncated and FALSE returned.

DBFWriteDoubleAttribute()

```
int DBFWriteDoubleAttribute( DBFHandle hDBF, int iShape, int iField,
                             double dFieldValue );
```

hDBF: The access handle for the file to be written, as returned by DBFOpen(), or DBFCreate().

iShape: The record number (shape number) to which the field value should be written.

iField: The field within the selected record that should be written.

dFieldValue: The floating point value that should be written.

The DBFWriteDoubleAttribute() function is used to write a value to a numeric field (FTInteger, or FTDoube). If the write succeeds the value TRUE will be returned, otherwise FALSE will be returned. If the value is too large to fit in the field, it will be truncated and FALSE returned.

DBFWriteStringAttribute()

```
int DBFWriteStringAttribute( DBFHandle hDBF, int iShape, int iField,
                             const char * pszFieldValue );
```

hDBF: The access handle for the file to be written, as returned by DBFOpen(), or DBFCreate().

iShape: The record number (shape number) to which the field value should be written.

iField: The field within the selected record that should be written.

pszFieldValue: The string to be written to the field.

The DBFWriteStringAttribute() function is used to write a value to a string field (FString). If the write succeeds the value TRUE will be returned, otherwise FALSE will be returned. If the value is too large to fit in the field, it will be truncated and FALSE returned.

DBFWriteNULLAttribute()

```
int DBFWriteNULLAttribute( DBFHandle hDBF, int iShape, int iField );
```

hDBF: The access handle for the file to be written, as returned by DBFOpen(), or DBFCreate().

iShape: The record number (shape number) to which the field value should be written.

iField: The field within the selected record that should be written.

The DBFWriteNULLAttribute() function is used to clear the indicated field to a NULL value. In the .dbf file this is represented by setting the entire field to spaces. If the write succeeds the value TRUE will be returned, otherwise FALSE will be returned.

DBFClose()

```
void DBFClose( DBFHandle hDBF );
```

hDBF: The access handle for the file to be closed.

The DBFClose() function will close the indicated xBase file (opened with DBFOpen(), or DBFCreate()), flushing out all information to the file on disk, and recovering any resources associated with having the file open. The file handle (hDBF) should not be used again with the DBF API after calling DBFClose().

DBFIsRecordDeleted()

```
int DBFIsRecordDeleted( DBFHandle hDBF, int iShape );
```

hDBF: The access handle for the file to be checked.

iShape: The record index to check.

Returns TRUE (non-zero) if the record is marked for deletion, otherwise it returns FALSE.

DBFMarkRecordDeleted()

```
int DBFMarkRecordDeleted( DBFHandle hDBF, int iShape, int bIsDeleted );
```

hDBF: The access handle for the file.

iShape: The record index to update.

bIsDeleted: TRUE to mark record deleted, or FALSE to undelete it.

Returns TRUE on success, or FALSE on error.

DBFGetNativeFieldType()

```
char DBFGetNativeFieldType( DBFHandle hDBF, int iField );
```

hDBF: The access handle for the file.

iField: The field index to query.

This function returns the DBF type code of the indicated field. It will be one of:

- 'C' (String)
- 'D' (Date)
- 'F' (Float)
- 'N' (Numeric, with or without decimal)
- 'L' (Logical)
- 'M' (Memo: 10 digits .DBT block ptr)
- '' (field out of range)